

FLEXBOX & GRID ACTIVITY

Team Contributions, Reflection, Accessibility, and QA

Project	Flexbox vs Grid Learning Activity
Repository	https://github.com/benjaminkettytagoe-alu/Week-5-flexbox-grid-adv-frontend
Date	October 8, 2026

1. Team Contributions

Each member selected a task and pushed their work from their own GitHub account to the group's shared fork.

Member	Task	Deliverables	Branch
Prince Ndahiro	HTML Structure	Setting up HTML structure	prince/level-0-html
Benjamin Kettey-Tagoe	Setup & Level 1	Project setup (Sass config, reset, variables); Level 1 Flexbox and Grid layouts	bkt/initial-setup
Megane Keza Mwizerwa	Level 2	Multi-column layout (Flexbox vs 2-column Grid) with mobile stacking	megane/leve-2-multicolumn
Benjamin Niyomurinzi	Level 3	Complex layout (3-column Flexbox vs named Grid areas) with tablet and mobile rules	feature/level-3-complex
Chris Marcel Hakizimana	Level 4 (Flexbox)	Holy Grail layout: Flexbox engine, fixed header, collapsible sidebars, card grid	Chris/level-4-flexbox
Chris Marcel Hakizimana	Level 4 (Grid)	Holy Grail layout: CSS Grid with grid areas/subgrid, sticky positioning	feature/level-4-grid
Kabanda Gislain	Docs & Integration	Flexbox vs Grid reflection, cross-browser testing, accessibility audit, final build QA, this document	feature/docs-and-qa

The project uses one HTML page for all levels. The layout technique and level are chosen with URL parameters (for example ?layout=grid&level=3), and a small switcher in the corner links to all eight combinations. Common styles are separated from layout-specific styles. For example, the Level 4 Flexbox stylesheet imports the shared variables, reset, base Level 4 styles and Flexbox-specific styles:

```
@use 'abstract/variables';
@use 'abstract/reset';
@use 'levels/level4-base';
@use 'levels/level4-flex';
```

This keeps repeated styles out of the two layout files and lets Flexbox and Grid be tested separately.

2. Flexbox vs Grid Reflection

Which was easier to implement?

For simple, one-direction layouts Flexbox was easier. Level 1 needs only flex-direction: column and flex-grow: 1 on the main area to push the footer down. For full page structures with sidebars, Grid was easier, because we could describe the whole page in one place with grid-template-areas instead of nesting several flex containers.

Which required less code?

For the page-level structure, Grid needed less code. One Grid container defines the rows, columns and areas, while the Flexbox version needs nested containers, flex-basis values and order rules on the children. Flexbox was shorter for small components such as the navigation links and the header with logo and user menu, where gap and justify-content do the work in a few lines.

Which was more intuitive?

Flexbox is quicker to learn because you think along one axis. Grid has more concepts (tracks, areas, or units), but it becomes more predictable once understood. The clearest example is the Level 3 tablet layout. In our HTML the left sidebar comes before the main content, so Flexbox has to use order to move the sidebars below it, while Grid only redefines grid-template-areas inside a media query. Reading the Grid version feels closer to reading a picture of the page.

In what scenarios would we prefer one over the other?

We would choose Flexbox for navigation bars, button groups, rows of cards and centring content, where the content decides the size. We would choose Grid for page skeletons, dashboards and any layout that needs rows and columns controlled together. In real projects the two work best together: Grid for the overall page and Flexbox for the components inside it.

Overall

Flexbox and Grid are not competitors. They solve different problems, and the useful skill is matching the tool to the structure of the interface. This activity also showed that a good layout is more than a page that looks right: responsiveness, accessibility, maintainable CSS and testing all matter, and building the same layout twice made the trade-offs between the two systems easy to see.

3. Cross-Browser and Responsive Testing

Each browser was used to open all eight versions (Flexbox and Grid, Levels 1 to 4) and the results recorded below.

Browser	Desktop	Tablet (~768px)	Mobile (~375px)	Notes
Google Chrome				
Microsoft Edge				
Mozilla Firefox				

Checked in each browser: header, navigation, left sidebar, main content, right sidebar, six content cards, footer, sidebar collapse buttons, links and buttons, text readability, and no unexpected horizontal scrolling.

4. Accessibility Audit

The review of index.html found the following good practices:

Area	Finding
Semantic HTML	Uses <header>, <nav>, <main>, <aside>, <section>, <article> and <footer> instead of generic <div> wrappers for page regions.
Navigation label	<nav class="site-nav" aria-label="Main navigation"> names the navigation region.
Sidebar labels	Both sidebars use aria-label ("Left sidebar", "Right sidebar").
Sidebar controls	Buttons use aria-controls and aria-expanded; JavaScript updates aria-expanded when a sidebar is collapsed or expanded.
Keyboard use	Controls are native <a> and <button> elements, so they are reachable with Tab and the buttons activate with Enter or Space.
Page basics	lang="en" and the responsive viewport meta tag are present.

Recommended improvement

The sidebar buttons currently read only "Toggle". A more descriptive label helps screen-reader users know which sidebar a button controls, for example "Toggle left sidebar" and "Toggle right sidebar" (or an aria-label with the same text).

5. Final Build Quality Assurance

Area	Check	Result
HTML	lang attribute, viewport meta tag, semantic elements, labelled navigation and sidebars	Confirmed in index.html
CSS / SCSS	Shared variables, reset and level-specific styles separated into partials	Confirmed (Sass structure)
Layouts	Flexbox and Grid versions selectable for Levels 1 to 4 through the switcher	Confirmed in index.html
Functionality	Sidebar buttons have JavaScript handlers that update aria-expanded	Confirmed in index.html
Responsiveness	Mobile, tablet and desktop widths; no unwanted horizontal scrolling; sidebars behave on small screens	See Section 3